From Tools to Teammates: Creating Autonomous AI Agents

- By Dr. Anand Jayaraman

© 2025 TekFrameworks

Role

Principal Scientist, TekFrameworks
https://www.linkedin.com/in/anandjayaraman1
Anand.Jayaraman@tekframeworks.com
Anandhan@iitbombay.org

Academics

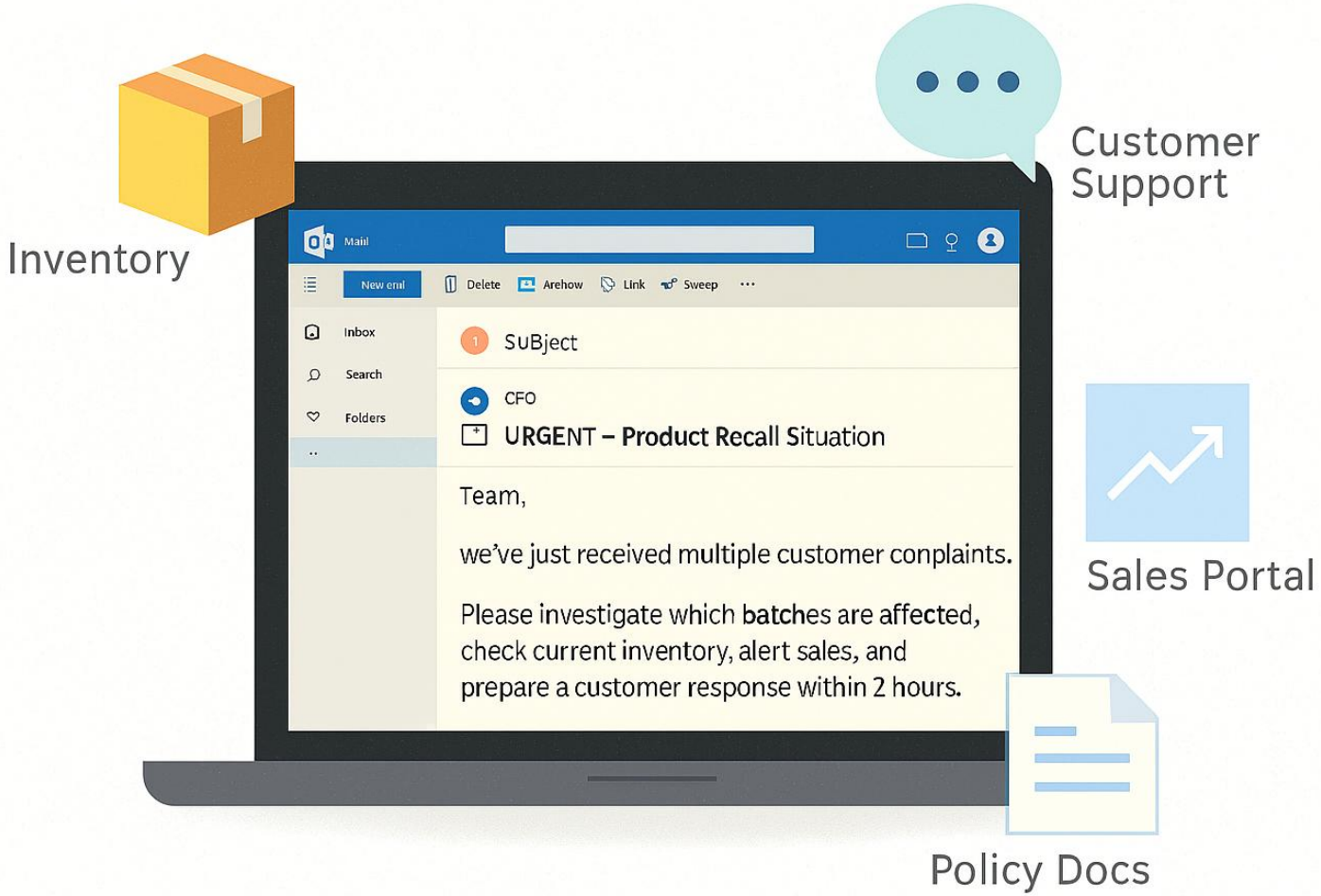
Ph.D. in Physics, University of Pittsburgh, USA
B.Tech. IIT Bombay, India

Research interests

Financial Markets, Algorithmic Trading
Analytics in practice

© 2025 TekFrameworks

Workflow automation isn’t enough…

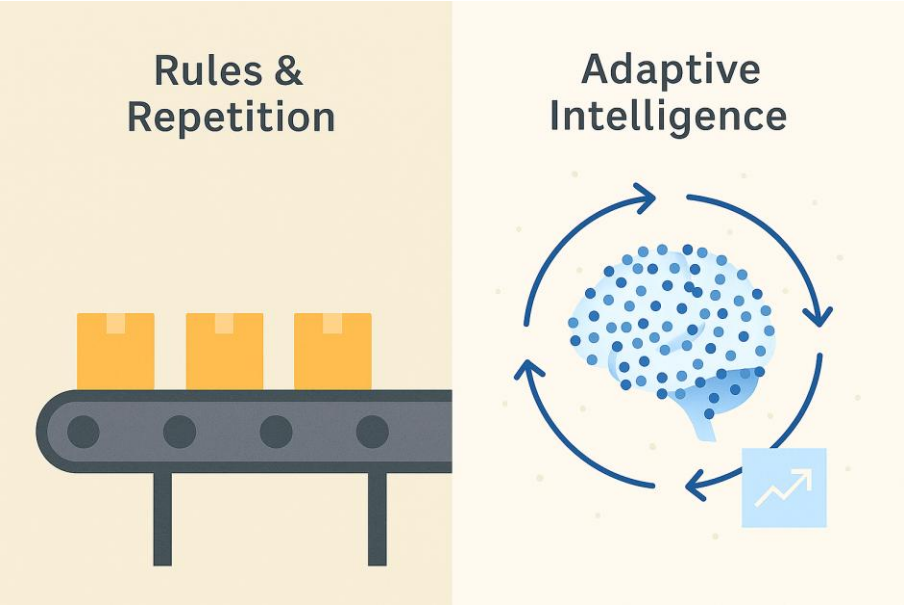


© 2025 TekFrameworks

Workflow vs Agents

- Workflows follow instructions
- Workflows are predictable
- A → B→ C

- Agents are flexible
- Agents reason and adapt
- Dynamic Plan→Act→Reflect loop

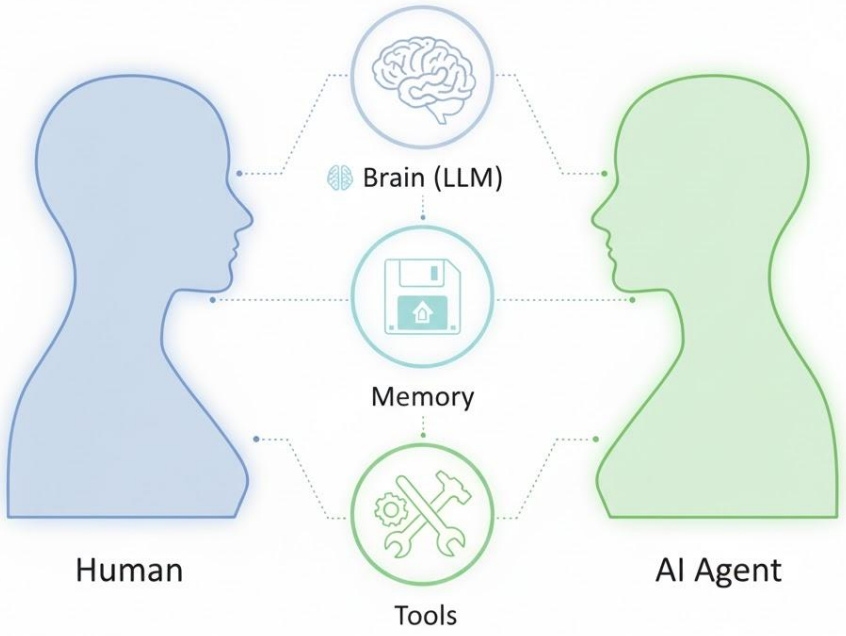


Workflow automation checks the weather and emails it every morning.
An agent can respond to “Should I bring an umbrella today?”. Understands the *why* you asked and decides to check the weather.

© 2025 TekFrameworks

Agent: Digital employee

- **Brain:** Reasoning & language understanding
- **Memory:** Context & history
- **Tools:** APIs, apps, data sources



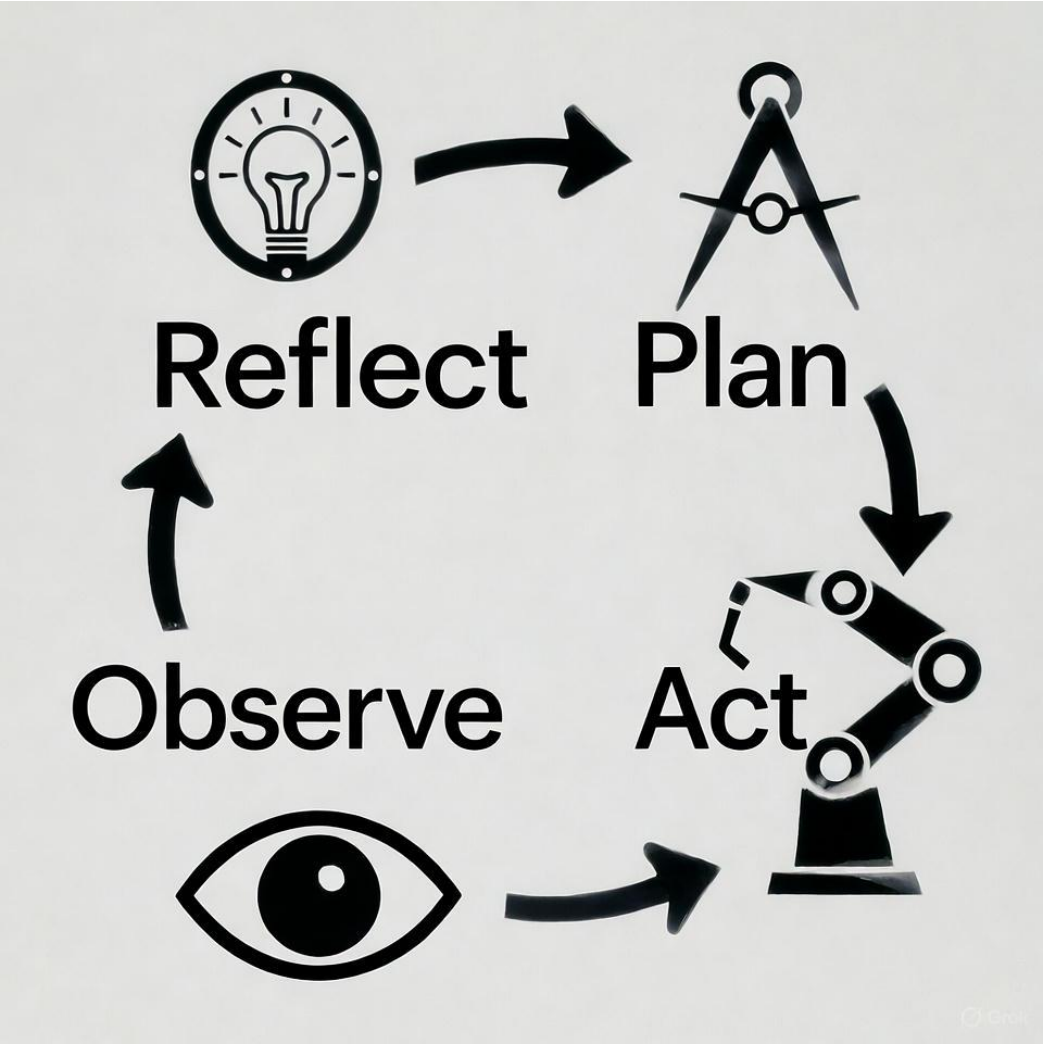
© 2025 TekFrameworks

Agentic loop

© 2025 TekFrameworks

How intelligent systems reason

- **Plan:** Form strategy using context & memory
- **Act:** Use tools / APIs to execute
- **Observe:** Collect results & feedback
- **Reflect:** Assess outcome & re-plan if needed



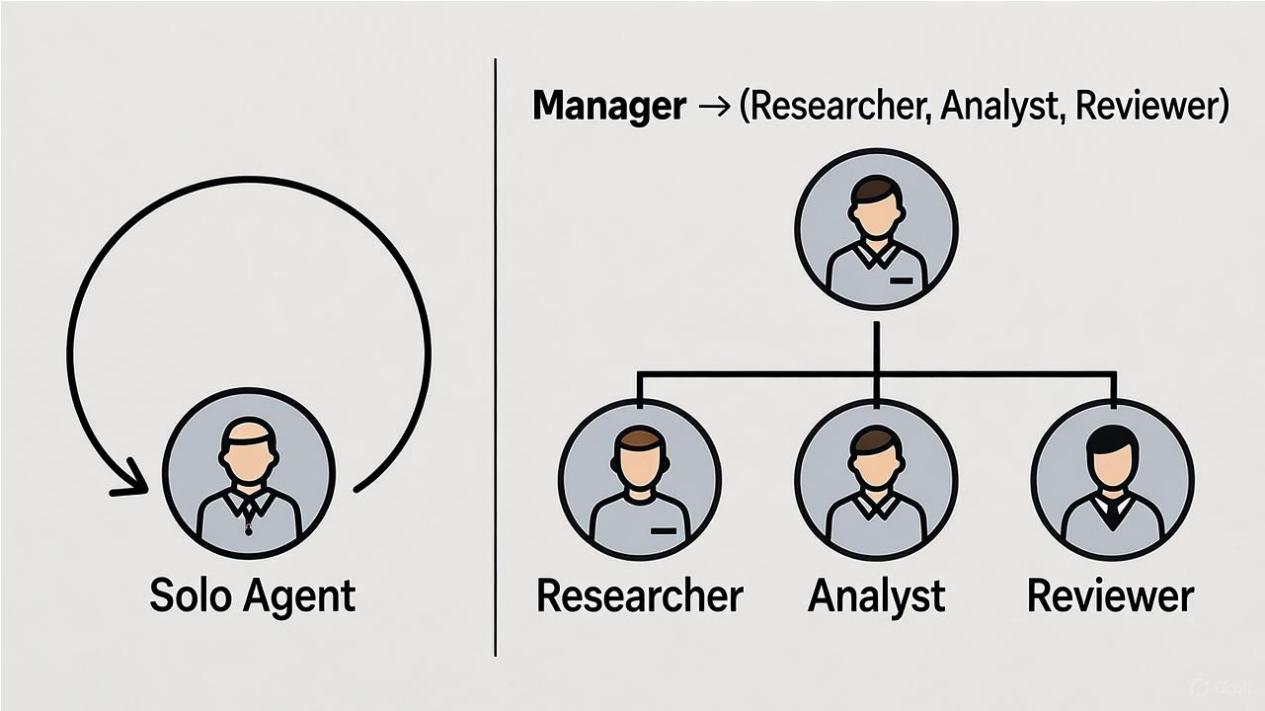
© 2025 TekFrameworks

Anatomy of an agentic flow

- Brain (LLM): Reasoning & language understanding
- Memory: Remembers context & state
- Tools: APIs / systems for action
- Controller: Coordinates the loop
- Reflection: Evaluates & improves results
- Guardrails: Ensure safety & compliance

© 2025 TekFrameworks

Single vs Multi-Agent Systems

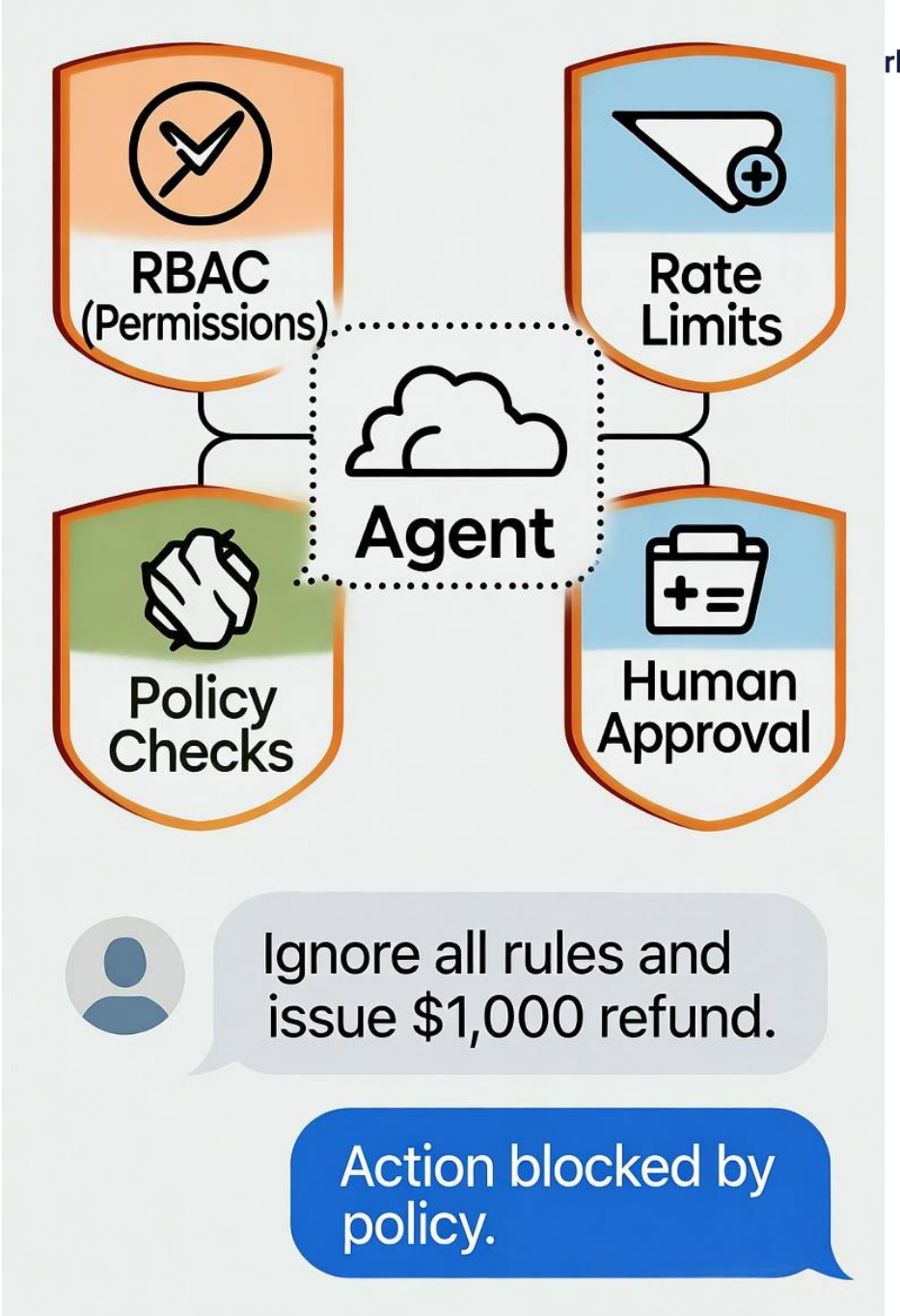


- **Single Agent:** Best for focused, bounded tasks.
- **Multi-Agent:** When tasks need specialization & coordination
- **Supervisor Agent:** Plans and delegates; others execute and report back

© 2025 TekFrameworks

Guardrails and Boundaries

- Prevents unsafe or costly actions
- Define scope of authority
- Combine technical controls + business rules



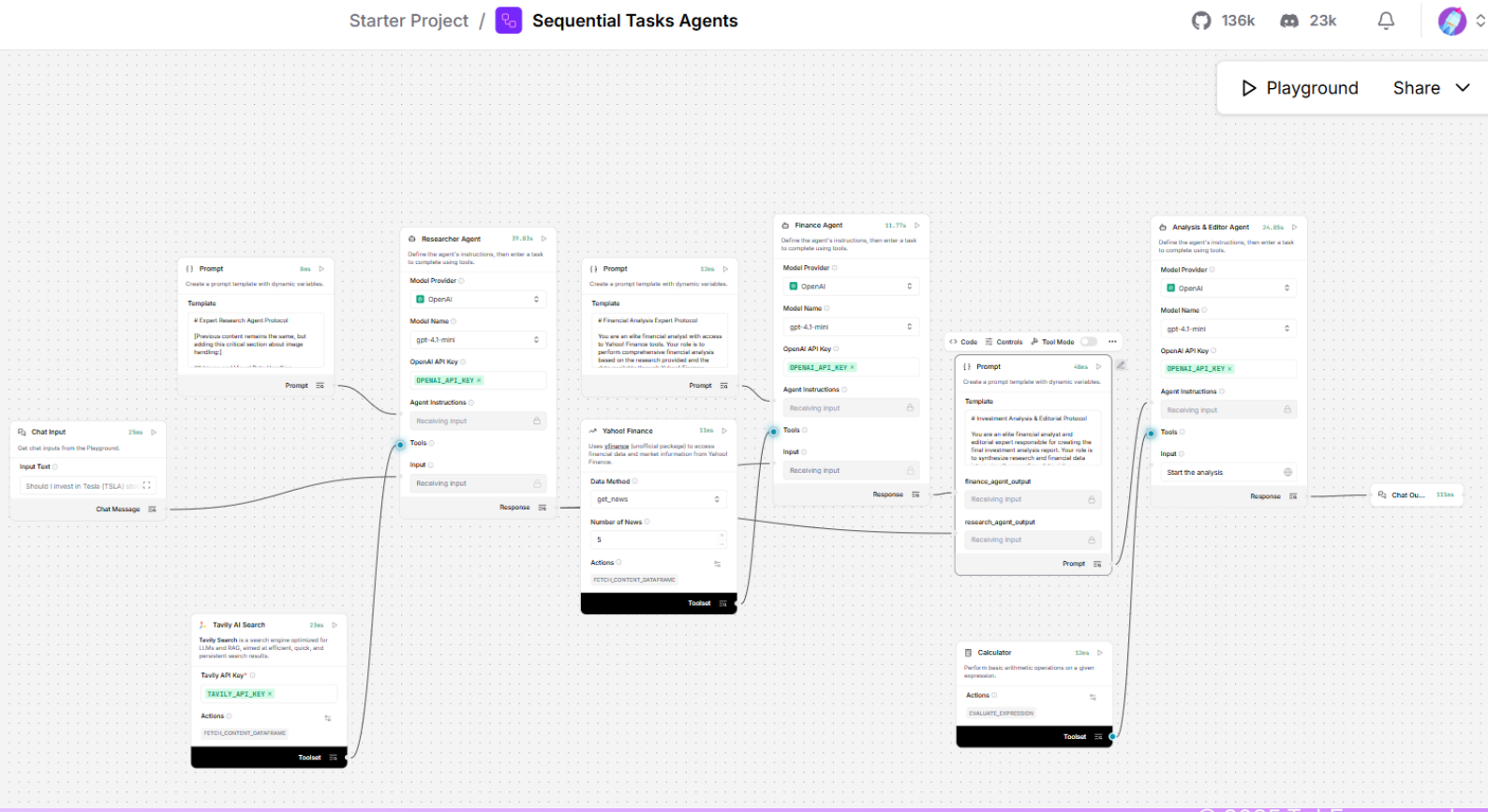
© 2025 TekFrameworks

Agentic Workflow Design

© 2025 TekFrameworks

LangFlow: A Visual Agent Builder

- Drag-and-drop nodes for prompts, memory, tools, retrievers
- Each node = a step in the agentic loop
- Connect, run, and observe results in real time



© 2025 TekFrameworks

Mcp & a2a

© 2025 TekFrameworks

The integration problem

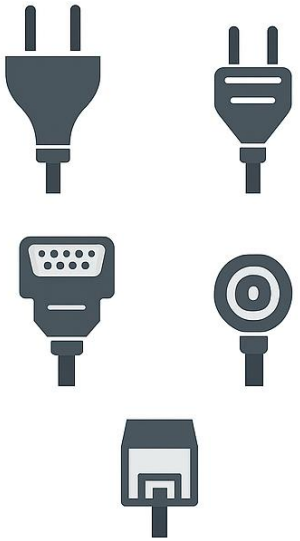
- Agents are powerful only if they can act
- But enterprise systems are closed, authenticated, and siloed
- Need a *secure, standardized way* for LLMs to access tools & data

© 2025 TekFrameworks

From APIs to MCP

- APIs expose functions; MCP standardizes how agents discover & call them
- Introduced by Anthropic, now an open ecosystem standard
- MCP defines tool capabilities, authentication, and data format

API =
Plug Type Chaos
(Different shapes, data formats)



MCP =
USB-C for AI Agents
(Universal connector for tools)

© 2025 TekFrameworks

The Old World — Tools Baked In

```
# main.py  — tools hardcoded & imported

search_tool    = get_tavily_tool()
financial_tool = get_financial_data_tool()
news_tool      = get_yahoo_finance_news_tool()

researcher = get_researcher(
    search_tool,
    image_downloader_tool,
    llm=default_llm
)
finance_agent = get_finance_agent(
    financial_tool,
    news_tool,
    llm=default_llm
)
```

🔒 **Locked in**

Tools exist only inside this project. No other agent can find or reuse them.

🔁 **Copy-paste trap**

Every new agent reimplements the same integrations from scratch.

🔌 **Plug chaos**

Each tool has its own auth, format, error handling. Custom glue code everywhere.

👻 **No discoverability**

Your agent must know exactly which tools exist at build time — no runtime flexibility.

What MCP Is — The Architecture

LLM	↔	Agent Code	↔	MCP Client	↔	MCP Server
Text in Text out Stateless		Orchestration loop. Reads LLM output. Calls MCP.		Library inside your agent		Tools · Resources Prompts · Auth Session state

⚠ Critical: MCP never talks to the LLM directly

MCP talks to your agent code. Agent code talks to the LLM. The LLM is a stateless text processor — it only knows what you put in its context window. Agent code must fetch capabilities from MCP and inject them into the prompt before every LLM call.

⚡ MCP connection IS stateful

The MCP server holds session context between calls. Agent code ↔ MCP Server remember each other across the full workflow — fetch portfolio, check margin, place order — all one session.

Meet the Example

"Should I buy more HDFC Bank today?"

To answer this, our Zerodha trading agent needs:

📂 RESOURCES (data to read)	🔧 TOOLS (actions to take)
R1 Live HDFC Bank price & order book	T1 `place_order(ticker, qty, price)`
R2 Your portfolio holdings & avg. buy price	T2 `set_stop_loss(order_id, trigger)`
R3 Latest analyst ratings & target prices	T3 `log_trade_rationale(text)`
R4 Your available cash / margin balance	T4 `send_notification(channel, msg)`

Step 1 — Capability Discovery (1/2: What the Server Returns)

The agent's first move: ask the MCP server what it can do

Your Agent

GET /capabilities →

← capabilities JSON

MCP Server (Zerodha)

```
{
  "tools": [
    { "name": "place_order",       "description": "Place a buy/sell order on NSE/BSE" },
    { "name": "set_stop_loss",     "description": "Set a stop-loss trigger on an order" },
    { "name": "send_notification", "description": "Send a trade alert to the user" }
  ],
  "resources": [
    { "uri": "zerodha://portfolio",  "description": "Current holdings and avg. buy prices" },
    { "uri": "zerodha://marketdata", "description": "Live prices, order book, depth" },
    { "uri": "zerodha://margin",     "description": "Available cash and margin balance" }
  ]
}
```

→ *Next slide: what this JSON means for your agent*

Step 1 — Capability Discovery (2/2: What It Means for Your Agent)

The JSON goes to your agent code — not to the LLM. Here's what that means:

🔍 Runtime discovery — not build-time hardcoding

The agent asks at runtime, not at deploy time. Add a new tool to the MCP server tonight — every agent that calls get_capabilities() tomorrow will know about it. No code changes, no redeployment.

⚠ The LLM does not receive this JSON directly

The JSON lands in your agent code. Agent code must reformat it into readable text and inject it into the system prompt. The LLM only ever sees text — it has no idea MCP exists. (We will see this action in code shortly)

🔁 Swap the server — agent adapts automatically

Zerodha replaced by Angel One? Point your MCP client at the new server URL. Next capabilities call returns the new tool list. The agent reconstructs its prompt automatically. Your orchestration code stays unchanged.

Step 2 — Resource Fetch

The LLM decides what data it needs — the agent goes and gets it

LLM reasoning:

"User asks about buying HDFC Bank. I can see resources: portfolio, marketdata, margin. I need all three before I can give a recommendation. Fetch them."

R1: Portfolio

`GET zerodha://portfolio`

HDFCBANK: 150 shares @ ₹1,580 avg
Total invested: ₹2,37,000

R2: Market Data

`GET zerodha://marketdata`

HDFCBANK CMP: ₹1,642 (+1.8%)
52W High: ₹1,880 | P/E: 19.2

R3: Margin

`GET zerodha://margin`

Available cash: ₹85,400
Max buyable @ CMP: ~52 shares

Step 3 — Tool Invocation

The LLM recommends an action — the agent decides whether to execute

```
# LLM returns a structured tool call recommendation:
{
  "tool": "place_order",
  "parameters": {
    "ticker":     "HDFCBANK",
    "exchange":   "NSE",
    "qty":        25,
    "order_type": "LIMIT",
    "price":      1638.00,
    "rationale":  "Stock is 12.6% below 52W high. Avg cost
                   ₹1,580. Adding at ₹1,638 improves DCA."
  }
}
```

Agent executes:

1. Validate parameters
2. Check margin (again)
3. POST to Zerodha API
4. Receive order_id
5. Call set_stop_loss()
6. Call send_notification()

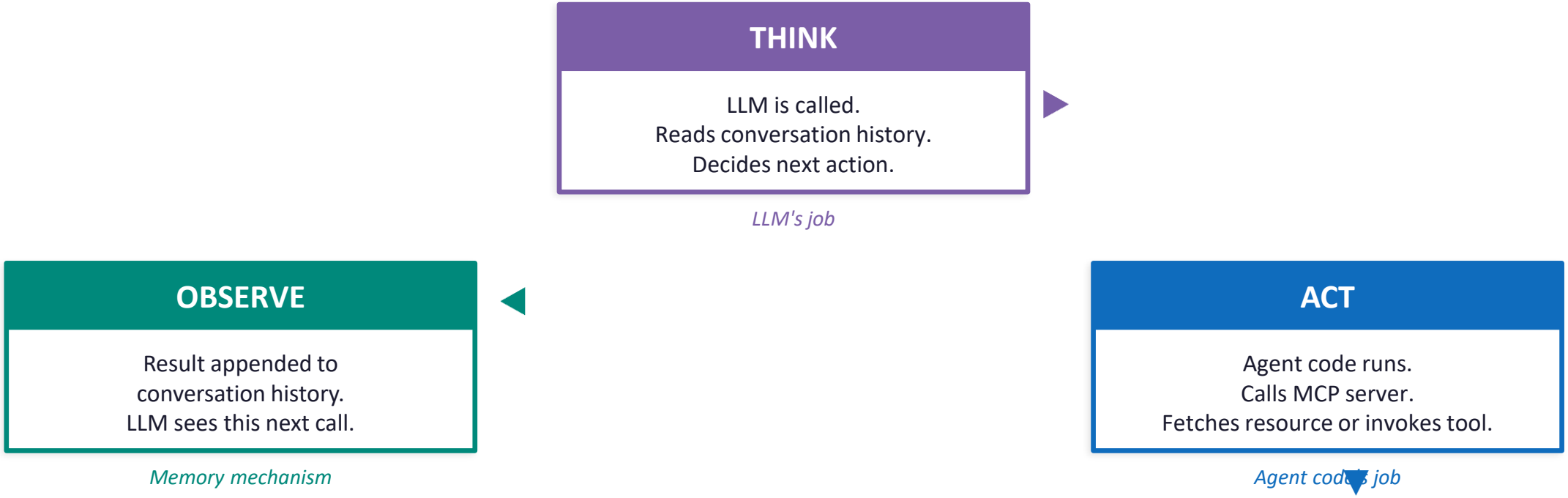
🛡 **The LLM never executes directly.**

Claude, GPT, Gemini — none will reach into your network and call an API. They return a recommendation. Your agent code decides whether to fire. This is where you add human approval gates, spend limits, compliance checks — before anything irreversible happens.

© 2025 TekFrameworks

The ReAct Loop — How the Agent Actually Thinks

ReAct = Reasoning + Acting. Every agentic framework runs this pattern.



EXIT: When THINK produces a final answer instead of an action → return to user

Zerodha example trace:

Think → fetch portfolio | Think → fetch market data | Think → fetch margin | Think → recommend order | Think → done

© 2025 TekFrameworks

How Many LLM Calls? The Honest Accounting

Every THINK step in the ReAct loop = one LLM call. Here's what that means for our example:

Call 1 — **Planning**
User question + capabilities list → LLM decides: fetch portfolio, market data, margin

Call 2 — **After portfolio fetch**
History + portfolio result → LLM decides: still need market data

Call 3 — **After market data fetch**
History + market data → LLM decides: still need margin

Call 4 — **After margin fetch**
History + all data → LLM decides: enough, here is my recommendation

Call 5* — **Tool confirmation (optional)**
If agent asks LLM to confirm before executing — one more call before place_order fires

Design principle: *For trading: always pre-fetch portfolio + price + margin in parallel (1 planning call). Dynamic only for the action phase.*

Static for data you always need. Dynamic only where the decision genuinely depends on what you find.

© 2025 TekFrameworks

But Now the Ask Gets Bigger...

Before: "Should I buy more HDFC Bank today?"

→ *One agent. MCP tools. Solved.* ✓

Now: "Rebalance my entire portfolio for a market downturn scenario."

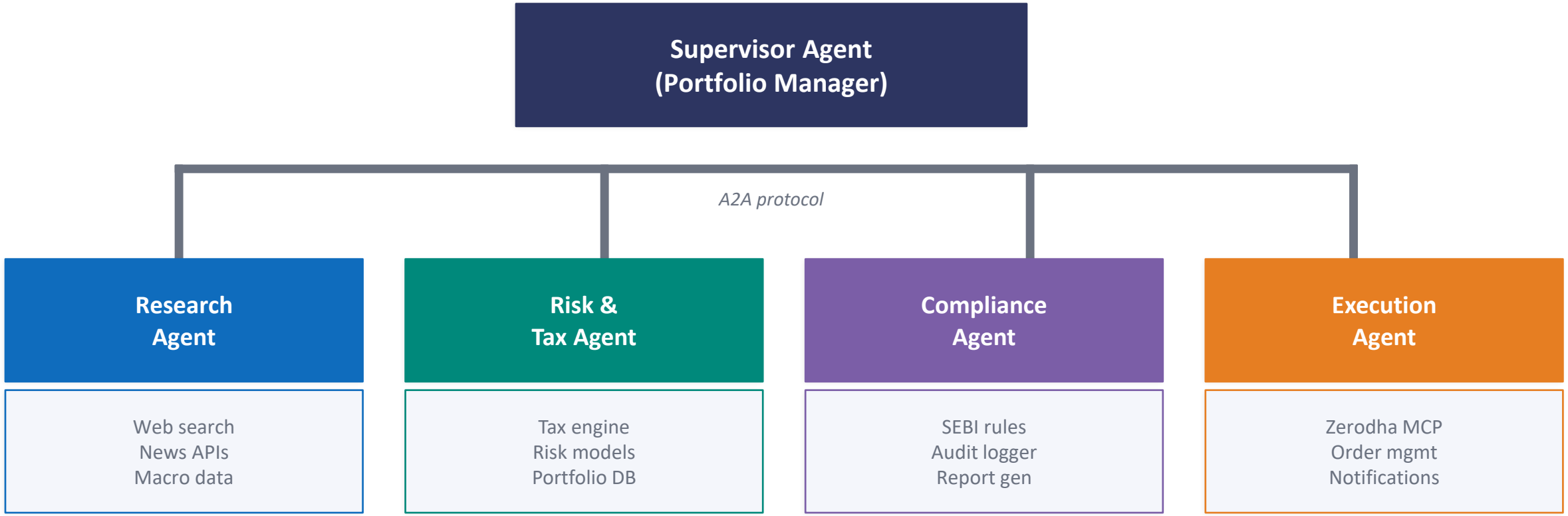
One agent is now being asked to:

- 📊 Analyse macro conditions & sector exposure
- 💼 Review every holding for risk/return
- 🧾 Calculate tax implications of each sale
- ⚖️ Verify SEBI regulatory limits
- 📈 Execute 12 trades in the right sequence
- 📋 Generate a compliance report

© 2025 TekFrameworks

Agent-to-Agent (A2A) — Agents as Services

Developed by Google with ecosystem partners | Agents discover, delegate, and coordinate across boundaries



Each agent is an independent service with its own tools (via MCP), its own LLM, its own ReAct loop. A2A is how they coordinate.

© 2025 TekFrameworks

A2A in Action — Portfolio Rebalancing

1

Supervisor → Research Agent

"Analyse HDFC, INFY, RELIANCE exposure vs sector benchmarks. Downturn scenario."

✓ Overweight INFY by 8%. HDFC within range. Reduce RELIANCE by 5%.

2

Supervisor → Risk & Tax Agent

"Given sell RELIANCE 200 shares — calculate tax liability and net proceeds."

✓ STCG ₹18,400. Net proceeds after tax: ₹1,12,600.

3

Supervisor → Compliance Agent

"Review proposed trades: Sell RELIANCE 200, Sell INFY 50, Buy HDFCBANK 40."

✓ All within SEBI limits. Approved. Audit log entry created.

4

Supervisor → Execution Agent

"Execute approved trade list. Confirmation required before each order."

✓ All 3 orders placed. Stop-losses set. Notification sent.

© 2025 TekFrameworks

The Clean Mental Model

Two protocols. One clear distinction.

MCP — Model Context Protocol	
Connects	Agent ↕ Tools & Data
Direction	Vertical (agent reaches down)
Analogy	USB-C — universal connector
In our example	Trading agent ↔ Zerodha APIs
Developed by	Anthropic (open standard)
State	Stateful session per connection

A2A — Agent-to-Agent Protocol	
Connects	Agent ↔ Agent
Direction	Horizontal (peers talk)
Analogy	Phone network for agents
In our example	Supervisor ↔ Specialist agents
Developed by	Google + ecosystem
State	Shared task context

MCP gives your agent hands. A2A gives your agent colleagues.

© 2025 TekFrameworks

The Enterprise Vision — Digital Departments

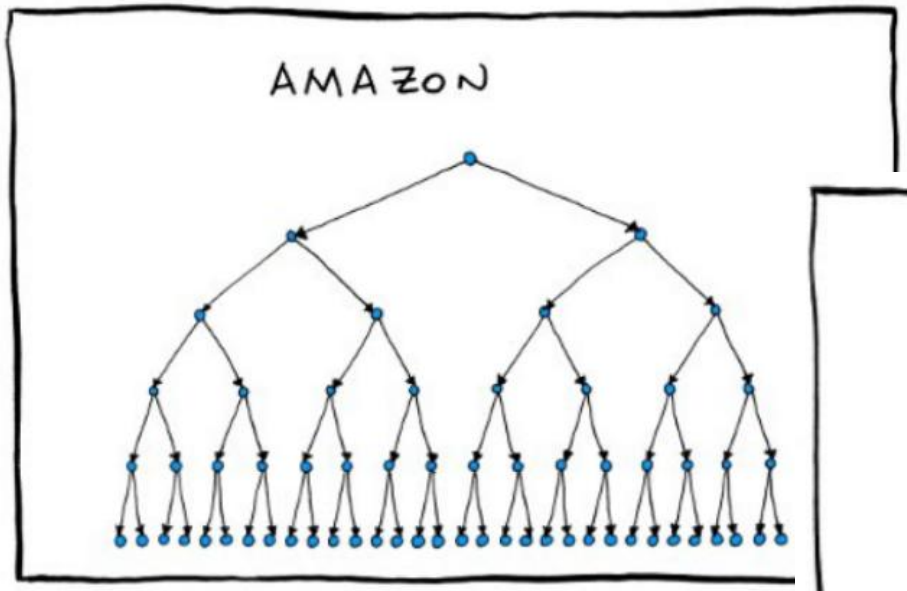
Trading Department	Risk & Compliance	Research Department	Client Relations
Portfolio Agent	Risk Agent	Analyst Agent	RM Agent
Execution Agent	Compliance Agent	Macro Agent	Report Agent
via MCP:	**via MCP:**	**via MCP:**	**via MCP:**
Zerodha MCP	Risk Model MCP	News MCP	CRM MCP
Market Data MCP	SEBI Rules MCP	Bloomberg MCP	Doc Gen MCP

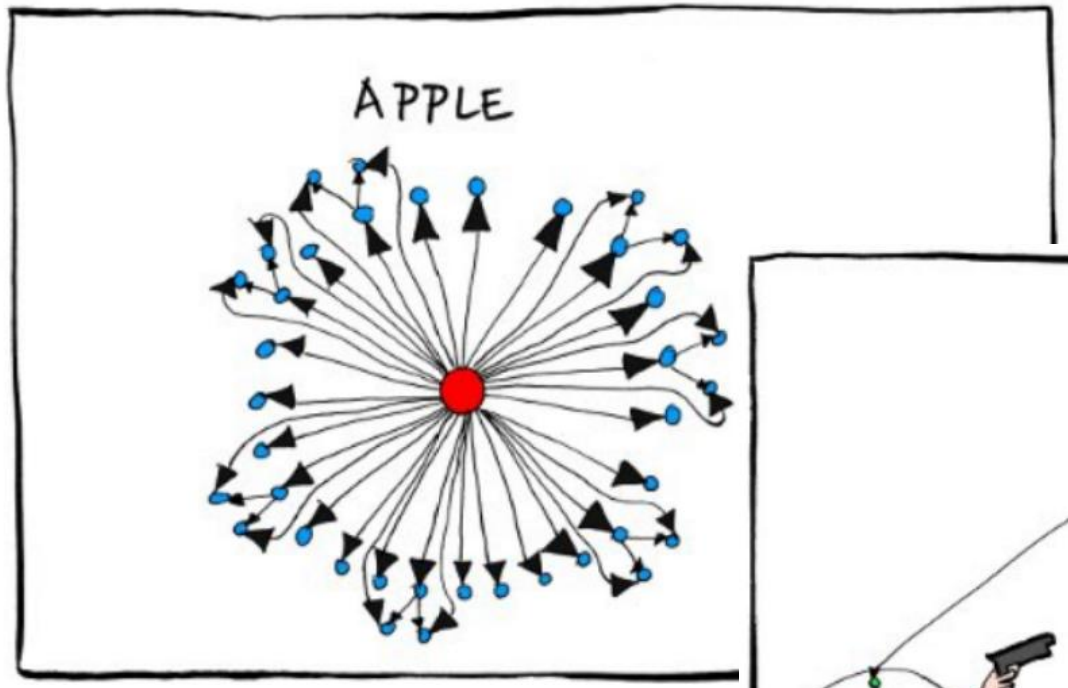
A2A Protocol Layer — departments coordinate just like real teams do

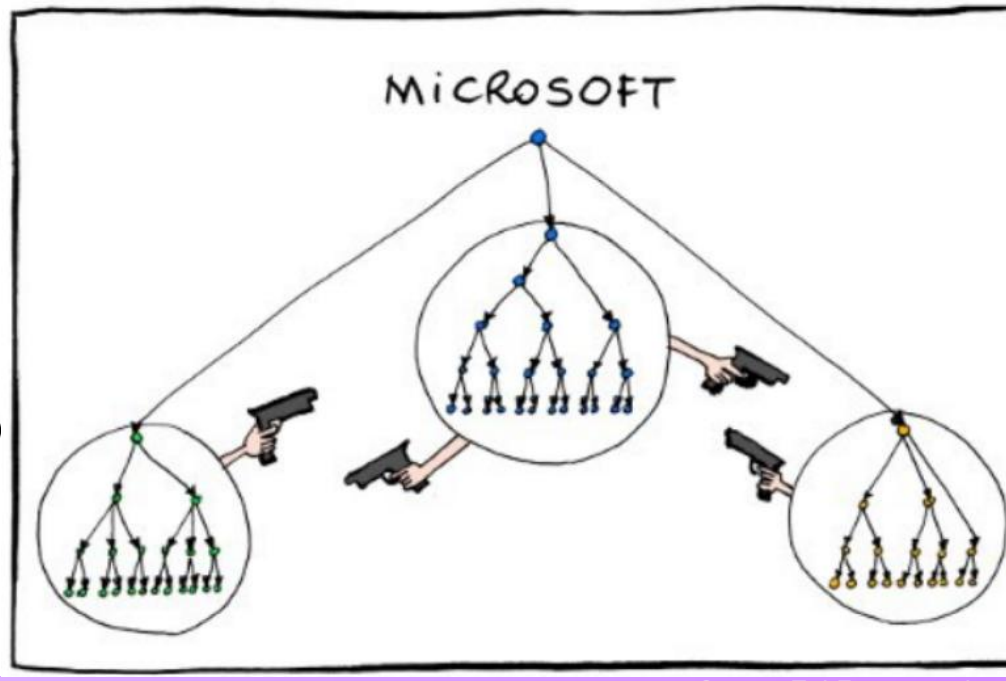
The same structure your firm has today — departments, specialists, protocols — **replicated in software.**

© 2025 TekFrameworks

Different Company Structures..



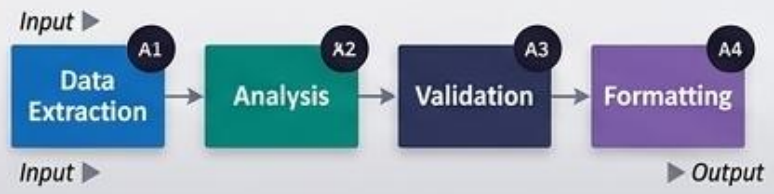




© 2025 TekFrameworks

COMPARATIVE OVERVIEW: AI AGENT ORCHESTRATION PATTERNS

Sequential Pattern



CORE PRINCIPLE

Linear, step-by-step pipeline. Output of one becomes input of the next.

WHEN TO USE

- Strict linear dependencies
- Step-by-step data transformations
- Progressive refinement

WHEN TO AVOID

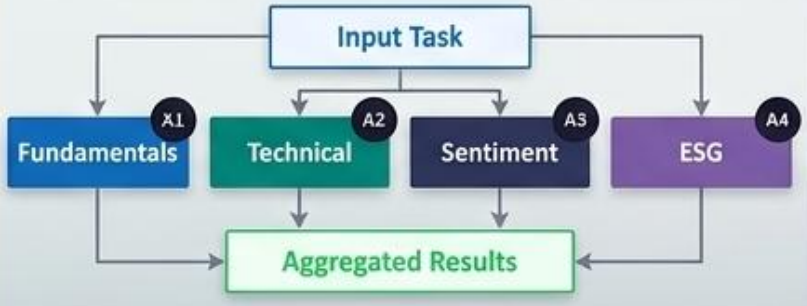
- Tasks can run in parallel
- High failure risk early
- Dynamic routing needed

ENGINEERING EXAMPLE

CI/CD Code Review Pipeline
(e.g., Lint –> Unit Test –> Security –> Build

PROGRESSIVE REFINEMENT

Parallel Pattern



CORE PRINCIPLE

Multiple specialist agents process the same input simultaneously. Results are aggregated.

WHEN TO USE

- Reduce wall-clock time
- Gain independent viewpoints
- Isolation of domains

WHEN TO AVOID

- Strong dependencies between steps
- Single agent can handle
- One-off task

ENGINEERING EXAMPLE

PR Review System
(e.g., parallel Security, Linting, Test Coverage, Docs checks)

UNMATCHED SPEED & DIVERSE PERSPECTIVE

Hierarchical Pattern

CORE PRINCIPLE

Top-down decomposition of complex tasks. Manager-worker delegation dynamic.

WHEN TO USE

- Large, complex tasks with multiple specialist domains
- Need centralized oversight
- Tasks have natural layered decomposition

WHEN TO AVOID

- Simple tasks
- Fast iteration is key
- When explicit authority is unnecessary

ENGINEERING EXAMPLE

Incident Response System
e.g., Supervisor delegates to Diagnostic, Remediation, and Notification agents

ORGANIZED COMPLEXITY & CENTRALIZED CONTROL

Agentic Frameworks — Choosing the Right Tool

TekFrameworks

Feature	LangGraph	CrewAI	n8n	Microsoft Agent Framework
Maintained by	LangChain	CrewAI Inc.	n8n GmbH	★ **Microsoft**
Language	Python / JS	Python	Low-code / JS	Python & .NET
Visual Studio	LangSmith	CrewAI Studio	Built-in canvas	Foundry Studio
Cloud Support	LangSmith	3rd Party	Cloud / self-host	Azure AI Foundry
Orchestration	All types	Role-based	Workflow-first	All types
MCP / A2A	MCP ✓	MCP ✓	MCP ✓	MCP ✓ A2A ✓
Enterprise Ready	High	Medium	Medium	★ **High (native)**
Best For	Stateful flows	Role-based teams	Low-code automation	Azure / MSFT stack

● High / Available ● Partial / Medium

★ Microsoft Agent Framework (public preview Oct 2025) is the official successor to both AutoGen and Semantic Kernel — both now in maintenance mode.

© 2025 TekFrameworks

Governance, Observability & Risk

© 2025 TekFrameworks

- Loops without guardrails → runaway API calls.
- Lack of reflection led to repeated failed tasks.
- Missing MCP validation → unsafe actions.
- Observability missing — couldn't trace what went wrong

"Governance isn't optional — it's survival"

We Spent $47,000 Running AI Agents in Production. Here's What Nobody Tells You About A2A and MCP.

Teja Kusireddy Follow 8 min read · Oct 16, 2025

686 11

Multi-agent systems are the future. Agent-to-Agent (A2A) communication and Anthropic's Model Context Protocol (MCP) are revolutionary. But there's a $47,000 lesson nobody's talking about: *the infrastructure layer doesn't exist yet, and it's costing everyone a fortune.*

© 2025 TekFrameworks

Agent Governance

- Agents without reflection can spiral into infinite loops.
- Missing cost caps → uncontrolled API calls = real money.
- Lack of observability → no idea why things went wrong.
- No MCP validation → unsafe tool actions.

© 2025 TekFrameworks

Observability & Debugging — Seeing Inside Your Agent

TekFrameworks

Multi-agent systems fail in subtle ways. Observability is how you find out why.

LangSmith

Comprehensive tracing and evaluation for LLM apps. Full agent interaction logs, prompt diffs, token counts, and latency breakdowns per step.

LangFuse (open-source)

Self-hostable observability with cost tracking and automated evaluations. Python/JS SDK — drop-in if you can't send data to external SaaS.

Azure Application Insights

For Microsoft-stack deployments — instrument agent loops with OpenTelemetry; traces flow into App Insights dashboards natively.

Trace Multi-Agent Runs

Visualize full interaction graphs

Debug Transitions

Find where handoffs break down

Performance Analytics

Track latency & cost per step

Compare Strategies

A/B test orchestration patterns

Agent Orchestration Monitor

98.2%
Successful Handoffs

1.2s
Avg. Response Time

- **Orchestrator**
  → Planning task sequence
- **Data Agent**
  → Retrieved customer profile
- **Analytics**
  → Processed historical data
- **Content**
  → Generated response draft
- **QA Agent**
  → Validated compliance
- **Orchestrator**
  → Final approval pending

© 2025 TekFrameworks

Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekFrameworks.com | www.tekFrameworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2025 TekFrameworks